

Developer Implementation Brief

Staged WordPress build: ordering, payments, store screens, analytics & scale

For the technical team · 17 June 2026 · Merges the client reference brief with an 11-agent code & live-site audit · Backup completed

1. Objective & MVP definition

Build a **staged DoughBoss Digital OS inside the existing WordPress site** without destabilising the live business. The MVP must prove, before any loyalty / full delivery / app-PWA / printer-automation / franchise tooling:

- Direct online ordering (pickup + optional delivery)
- **Store-specific menu & availability** (per-shop price & sold-out) — core, from day one
- Online payments (Stripe, AUD)
- A live **staff order screen** per shop
- **Conversion tracking** (treated as MVP, not a later add-on)

2. Current technical snapshot & known issues

Area	Observation
Platform	WordPress 7.0, PHP 8.2.31; Dough Boss child theme (classic); shops: Bankstown, Revesby, Roselands.
Plugin	DoughBoss plugin (was v2.0.0; now v2.1.0 after this engagement) with REST routes for config/menu/cart/checkout/lookup/admin status.
Content	43 menu-item posts + 3 location posts + 9 pages + 6 CF7 forms + 66 images (legacy/Toolset content — not yet wired into the ordering plugin's own item type).
Known bug	Menu taxonomy query uses <code>field=slug</code> but passes the category <i>name</i> → Flat Bread 11" renders empty despite having items.
Config	Ordering API returns USD , generic sizes/toppings, empty menu endpoint — treat as unfinished/demo.
Marketing	No SEO plugin; basic <code><head></code> ; 66/66 images missing alt text ; no confirmed GA4/Meta/TikTok stack.

3. Delivery principles

- **No live edits without backup + staging + owner approval**; keep the public site live throughout.
- **Separate display data from pricing/order logic** — no price in product titles; structured `base_price` + per-store override.
- **Store-specific ordering from day one**, even with one store live.

- Stripe hosted/Elements to minimise PCI; **server is the source of truth for totals** (never trust the browser).
- Log every order state change; non-dev admin controls for availability / prep-time / sold-out.
- Track events client- and server-side; minimise sensitive data; respect consent.

4. Target architecture

WordPress stays CMS/admin and system-of-record. Ordering logic lives in a dedicated plugin (see §5). The customer ordering app is mobile-first JS embedded in WP; the **staff screen is a separate protected per-store route, polling every 5–10s for the MVP**, with a managed push upgrade (Ably/Pusher) and customer live-tracking as a fast-follow — not a launch blocker. Stripe PaymentIntent + webhook. Custom tables for orders/items/status-logs/payments; CPTs only for marketing/menu editing. Conversion events fire client-side (via GTM) and server-side (Purchase/Lead on confirmation, off the order hooks).

5. Plugin structure & refactor-vs-extend decision

Decision: extend the existing DoughBoss plugin (already refactored to v2.1.0 with the order board, migration runner and transactional writes) rather than start a parallel `doughboss-ordering` plugin — lower risk, already delivering. Adopt the reference's module layout *within* it, and map the reference's `db-order/v1` route names to our `doughboss/v1` namespace (Appendix B).

```
includes/ Database/migrations · PostTypes · Rest/Public · Rest/Admin
          Payments/StripeService · Orders/OrderService
          Notifications/NotificationService · Marketing/EventService
public/   customer-app (menu/builder/cart/checkout) · staff order board
admin/    orders, order board, settings, (manager dashboard later)
```

Introduce provider interfaces (`PaymentGateway` , `Notifier` , `OrderRepository` , `FulfilmentChannel`) + PSR-4 namespace/DI as logic grows.

6. Data model (MVP)

Custom tables under the plugin. **v2.1.0** already added `eta_minutes`, `seen_at`, `acknowledged_at`, `accepted_at` + a status-history-ready structure and a migration runner.

Table	Key fields
<code>db_locations</code>	name, hours_json, pickup/delivery_enabled, prep_time_default, is_open
<code>db_menu_categories</code>	name, sort
<code>db_menu_items</code>	category_id, name, base_price, dietary_flags
<code>db_item_location</code>	item_id, location_id, price_override , sold_out (store-specific availability — P0)
<code>db_modifier_groups</code> / <code>db_modifiers</code>	min_select, max_select / price_delta
<code>db_orders</code> / <code>db_order_items</code>	+ shop_id, eta, payment fields / name_snapshot, modifiers_json (immutable price snapshot)
<code>db_order_status_log</code>	order_id, from, to, user_id, ts (audit trail)
<code>db_payments</code>	payment_intent_id, raw_event_id, state
<code>db_customers</code>	contact, marketing_opt_in
<code>db_coupons</code>	code, type, value (MVP offers)

7. REST endpoints (doughboss/v1)

Endpoint	Purpose	Status
GET <code>/config</code>	currency (→ AUD), fees	fix USD → AUD
GET <code>/locations</code>	shops + hours + open state	new
GET <code>/menu?location_id=</code>	per-store menu + availability	new (replaces demo)
POST <code>/cart/validate</code>	server-authoritative totals + price snapshot	harden
POST <code>/checkout (session)</code>	PaymentIntent; Idempotency-Key	idempotency v2.1.0
POST <code>/webhooks/stripe</code>	payment source of truth	new
GET <code>/order/{number}</code>	customer tracking (email/phone token)	harden + token
GET <code>/admin/orders</code>	staff order-board feed	v2.1.0
POST <code>/admin/order/{id}/ack</code> · <code>/accept</code> · <code>/status</code>	acknowledge · accept+ETA · advance	v2.1.0
GET <code>/admin/reports/daily</code>	daily sales/exports	P1

8. Payments

- Stripe **test mode first** — no live until every store passes test orders. Server-created PaymentIntent (AUD); **never trust browser totals**; immutable per-item price snapshot.
- **Webhook is the source of truth**. Payment states: `pending` / `paid` / `failed` / `refunded` / `partially_refunded`. Email only on paid.
- **Refund / cancel behind manager capability**, logged. PCI minimised (Elements/Checkout, SAQ-A). Review AU card-surcharging rules before launch.
- Fees: 1.7% + A\$0.30 domestic / 3.5% + A\$0.30 international (re-confirm before launch).

9. Staff order screen (KDS)

v2.1.0 shipped the polling board with New/Preparing/Ready lanes, looping audible alert until **Acknowledge**, one-tap Accept(+ETA)/status, large touch targets, and a low-privilege `doughboss_kitchen` role. Reference additions to fold in:

- Auth via staff login / **per-store device token** — no public screens; scope each device to its store.
- Card shows name / phone / pickup time / items / modifiers / notes / payment status.
- **Fail-safe backup email + optional SMS to the manager** if an order isn't acknowledged; tablet kept awake/powered.
- **Lost-connection recovery** (reconcile via `since=` cursor on reconnect); **min 20 test orders per store** before go-live.

10. Analytics & conversion tracking (MVP)

- GTM as the central client-side layer; server-side Purchase/Lead off the order hooks for reliability (hybrid). Dedup by `event_id = order_id`.
- GA4 + Meta Pixel + TikTok Pixel event map: `ViewContent / menu_view / item_view / AddToCart / InitiateCheckout / Purchase / Lead`; **verify each fires once with correct values**.
- Meta CAPI = Phase 2; Search Console + sitemap; consent-gated (AU Privacy Act).

11. SEO / Local SEO

- Install an SEO plugin (AIOSEO/RankMath/Yoast) non-destructively: metadata, schema, redirects, sitemap.
- Fix 66/66 missing image alt text; per-store `LocalBusiness/Restaurant` schema + suburb pages (Bankstown/Revesby/Roselands); update privacy policy.

12. Security / privacy / compliance

- Sanitise input / escape output; capability checks; nonces / app-auth on protected routes; **rate-limit checkout & order-tracking endpoints**.
- No raw card data; minimal PII; log staff actions; HTTPS / no mixed content; **confirm restorable backup before live payments**.

13. QA test plan

Test groups: menu rendering (incl. the taxonomy bug), cart/pricing (**browser cannot alter total**), payments (success/fail/abandoned/**duplicate webhook/refund**), staff screen (incl. **lost-connection recovery**), location logic (**closed store can't receive**), emails/notifications, marketing events (fire once, correct values), mobile devices, business-hours load, content/SEO.

14. Reporting & exports

MVP = admin list tables + **daily CSV exports** (`/admin/reports/daily`); manager dashboard (orders, revenue, top items, cancellations, per-store, lead reporting) post-MVP; accounting export (Xero/Zoho) later.

15. Acceptance criteria / launch gates

- Backup + restore verified known-good; owner signs off item names/descriptions/prices/availability.
- Stripe test passes; **each store completes ≥20 test orders**; refund/cancel documented.
- Privacy/terms updated; conversion events verified in GA4/Meta/TikTok.
- **Soft-launch ONE store before all-store rollout**.

16. Merged backlog (= shipped in v2.1.0)

Pri	Item
P0	Live order board (polling) · kitchen role · transactional order create + rollback · idempotent checkout · migration runner · <code>/admin/orders</code> , <code>/ack</code> , <code>/accept</code>
P0	Staging env + documented rollback; backup-restore verification gate
P0	Fix menu taxonomy bug (Flat Bread 11"); normalise menu (prices out of titles) ; AUD/config cleanup (remove demo USD data)
P0	Store-specific availability (<code>db_item_location</code> <code>price_override/sold_out</code>) + per-store menu API
P0	Server-authoritative cart totals + immutable price snapshot; rate-limit checkout/tracking
P0	Stripe PaymentIntent test-mode + webhook (source of truth); closed-store/hours guard
P0	Staff screen: store device token, manager email/SMS fail-safe, lost-connection recovery
P0	GA4 + Pixel core event map (fire-once verified); privacy-policy update; 20 test orders/store + single-store soft launch
P1	Abyl/Pusher realtime upgrade + customer live tracking; refund/partial-refund + cancel (manager cap)
P1	<code>/admin/reports/daily</code> + CSV exports; SEO plugin + Search Console + Local SEO pages + alt-text fix
P1	Meta CAPI + TikTok server events; coupons (MVP); PrintNode thermal print; multi-shop routing; consent management
P2	Loyalty/wallet; gift cards; advanced coupons/bundles; full PWA install/offline; delivery dispatch + tracking; franchise portal + multi-entity reporting; server-side AI reporting

17. Deferred (post-MVP)

Thermal auto-print · delivery dispatch/tracking · loyalty/wallet · gift cards · PWA install/offline · advanced coupons/bundles · franchisee portal & multi-entity reporting · warehouse + AI reporting.

18. Estimate & team

Phase 1 ≈ **169–344 hours** across 9 workstreams (Ordering API 35–70 · Customer UI 30–60 · Staff screen 24–50 · payments, data model, SEO/analytics, QA, etc.). Team model: **10 specialist agents + 2 managers** (Tech + Growth/Ops), already used to produce this brief.

Appendix A — what shipped in v2.1.0

Real-time Live Order Board (polling KDS) · `doughboss_kitchen` role + `manage_doughboss_kds` · transactional `create()` + rollback · order-number widening + retry · `/checkout` Idempotency-Key · versioned migration runner (DB 1.1.0) · new endpoints `/admin/orders`, `/admin/order/{id}/ack`, `/admin/order/{id}/accept` · ETA + seen/acknowledged/accepted timestamps · `doughboss_order_accepted` hook. Branch `claude/funny-goodall-gsoog4`, PR #2.

Appendix B — route mapping (reference → ours)

db-order/v1/config|locations|menu|cart/validate|checkout/session|webhooks/stripe|orders/{n}|
staff/orders|staff/orders/{id}/status|admin/reports/daily ↔ doughboss/v1/config|locations|
menu|cart/validate|checkout|webhooks/stripe|order/{n}|admin/orders|admin/order/{id}/status|
admin/reports/daily.